

Contents

guide Evaluating Large Language Models Trained on Code	1
0. 先给结论	2
1. 论文出现时的语境	2
2. 论文的 story	3
3. HumanEval 到底是什么	3
4. functional correctness 为什么重要	4
5. pass@k 的直觉	4
6. pass@k 的数学	5
7. 为什么不能偷懒用 naive 公式	6
8. 评测管线图	6
9. sandbox 为什么不是小事	7
10. 模型和训练数据	7
11. token 级别怎么想	8
12. 主结果: Codex 在 HumanEval 上发生了什么	8
13. 温度和采样数量	9
14. 候选排序: 没有测试时怎么办	9
15. BLEU 为什么靠不住	10
16. 与 GPT-Neo、GPT-J、TabNine 的对比	10
17. APPS 实验说明了什么	10
18. Codex-S: 为什么监督微调有用	11
19. Codex-D: 反过来从代码写 docstring	12
20. 失败模式: 长链操作和变量绑定	13
21. 安全和 broader impacts	13
22. 论文证据链条	14
23. 这篇论文没有证明什么	15
24. 和本仓库代码的对应关系	15
25. 本仓库最小实验 1: pass@k 手算	16
26. 本仓库最小实验 2: 隐藏测试抓边界条件	17
27. 本仓库最小实验 3: pass@1 和 pass@k 的差别	17
28. 本仓库最小实验 4: 从 HumanEval 到 SWE-Bench	17
29. 如果你要用 AI agent 学这篇	18
30. 对现在的意义	18
31. 读完必须能回答	19
32. 你下一步该怎么学	19

guide_Evaluating Large Language Models Trained on Code

原论文: Evaluating Large Language Models Trained on Code

本地原文 PDF: 已入库,同目录文件 01_evaluating_large_language_models_trained_on_code.pdf

作者: Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor

Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, Wojciech Zaremba

年份: 2021

本 guide 目标: 让你不只是知道 HumanEval 和 pass@k 的名字, 而是理解为什么这篇论文把代码模型评测的重心从 token loss/BLEU 拉到可执行功能正确性, 并且知道如何用本仓库代码复现它的最小机制。

0. 先给结论

这篇论文的核心贡献可以压缩成一句话:

代码模型不能主要用文本相似度评价, 而应该让模型写出可执行函数, 再用隐藏单元测试判断功能正确; 当模型可以多次采样时, 用 pass@k 衡量“至少有一个样本正确”的概率。

它的历史意义有三层:

1. 它发布并使用 HumanEval: 164 个手写 Python 函数题, 每题有函数签名、docstring、参考实现和隐藏单元测试, 平均每题 7.7 个测试。
2. 它把评测指标定为 functional correctness: 不是“生成文本像不像参考答案”, 而是“运行以后是否满足规格”。
3. 它把多采样能力变成 pass@k: 现实中程序员会尝试、修复、筛选, 模型也可以生成多个候选; pass@k 就是这个使用方式的评测语言。

从今天回看, HumanEval 已经显得小、静态、偏函数题, 也很容易被污染。但它给了后续代码模型、agent benchmark 和 execution-based eval 一套共同语言。后来的 MBPP、APPS、LiveCodeBench、BigCodeBench、SWE-bench、BFCL、WebArena 都可以看成是在补它的边界。

1. 论文出现时的语境

2021 年的背景是:

- GPT-3 已经说明大型自回归语言模型会涌现出许多自然语言能力。
- GitHub 上有大量公开代码, 代码也是一种 token 序列, 理论上可以拿来做 next-token prediction。
- 但“会写代码”和“在代码上有低 loss”不是一回事。
- 当时很多代码生成评价仍依赖 exact match、BLEU、CodeBLEU 或人类主观判断。

这些评价对代码有天然问题。一个函数可以有很多等价写法:

```
def add(a, b):  
    return a + b  
  
def add(a, b):  
    total = a  
    total += b  
    return total
```

文本上它们不同, 功能上它们相同。如果用 BLEU 或 reference overlap 打分, 模型可能被惩罚; 如果只看困惑度, 模型可能学会写“看起来像代码”的字符串, 却不一定能通过测试。

所以这篇论文问了一个更工程化的问题:

如果用户给一个函数签名和 docstring, 模型能不能写出真的能运行、能通过单元测试的函数体?

这就是从语言建模走向程序综合的关键转向。

2. 论文的 story

论文的 story 可以按这条链读:

大模型已经能预测代码 token

->

但 token loss 和 BLEU 不能代表程序正确

->

构造 HumanEval 手写题集

->

让模型从 docstring 生成函数体

->

在 sandbox 里执行隐藏单元测试

->

用 pass@k 衡量多采样下至少一个答案正确的概率

->

分析规模、温度、采样数量、监督微调和失败模式

->

讨论代码生成的安全、经济、法律和过度依赖风险

你读这篇论文时不要把它当成“Codex 模型介绍”。它更像是一篇评价范式论文: 模型是重要对象, 但真正留下来的基础设施是 HumanEval、functional correctness、pass@k、sandbox execution 和风险分析。

3. HumanEval 到底是什么

HumanEval 是作者手写的 164 个 Python 编程问题。每个问题包含:

- 函数签名, 例如 `def is_prime(n):`
- docstring, 用自然语言描述函数应该做什么。
- 函数体位置, 交给模型补全。
- 多个单元测试, 用来判断生成结果是否正确。
- 参考实现, 但评价时不是拿生成结果和参考实现做文本匹配, 而是运行测试。

论文强调“手写”很重要。因为训练数据来自大量 GitHub 代码, 公开仓库里已经有很多竞赛题、面试题和答案。如果直接拿公开题库, 会有数据污染风险。手写题不能保证绝对新颖, 但至少降低了从训练集中直接复制的概率。

HumanEval 覆盖的能力包括:

- 语言理解: 能不能读懂 docstring。
- 简单数学: 能不能处理边界条件和数值关系。
- 算法与数据结构: 能不能写循环、排序、过滤、字符串处理。
- 规格遵循: 能不能正确绑定变量、操作和返回值。

一个 HumanEval 样本在概念上长这样:

```
def count_vowels(s):  
    """  
    Return the number of vowels in s.  
    Vowels are a, e, i, o, u.
```

```
"""  
# model writes body here
```

评测时，模型可能生成:

```
def count_vowels(s):  
    return sum(1 for ch in s if ch in "aeiou")
```

然后隐藏测试可能包含:

```
assert count_vowels("hello") == 2  
assert count_vowels("xyz") == 0  
assert count_vowels("AEIOU") == 5
```

如果测试考虑大小写，而模型没有处理大写，样本就会失败。这就是 execution-based eval 的力量: 它可以抓住文本相似度看不出来的语义错误。

4. functional correctness 为什么重要

论文第 2.1 节的中心主张是: 代码生成应该优先用功能正确性评价。

BLEU 的逻辑是“生成文本与参考文本越像越好”。这在翻译里也有争议，在代码里更危险，因为代码存在大量语义等价变体:

```
return list(reversed(items))
```

和

```
items = items[:]  
items.reverse()  
return items
```

如果二者都满足函数规格，评测就不该因为 token 不像参考答案而扣分。

functional correctness 的基本流程是:

```
problem prompt  
-> model completion  
-> extract runnable code  
-> execute hidden tests  
-> passed or failed
```

它更接近真实开发。开发者不会问“这个函数和参考答案有多像”，而会问:

- 它能不能跑?
- 它在边界条件下对不对?
- 它有没有抛错?
- 它有没有违反安全约束?
- 它能不能和现有测试套件一起通过?

这也是后来 SWE-bench、LiveCodeBench、agent eval 共同继承的核心思想。

5. pass@k 的直觉

模型生成代码不是确定性解题器。给同一个 prompt，它可以采样多个候选:

```
sample 1: wrong
sample 2: wrong
sample 3: correct
sample 4: wrong
```

如果只看第一个样本，模型失败。如果允许生成 4 个，再由测试或人类筛选，模型成功。

所以 pass@k 表示：

对每道题采样 k 个候选，只要其中至少一个通过测试，就认为这道题在 pass@k 下被解决。

直觉上：

- pass@1 衡量“一次生成就对”。
- pass@10 衡量“给十次机会能不能出现一个对的”。
- pass@100 衡量“大采样预算下模型分布里是否含有正确程序”。

这很重要，因为代码模型的使用方式天然包含搜索。用户可能让模型重试，IDE 可能展示多个候选，agent 可能生成、运行、修复、再生成。pass@k 比 pass@1 更接近这种“采样加筛选”的过程。

6. pass@k 的数学

论文没有直接用“生成 k 个，看有没有一个对”来估计 pass@k，因为这样方差高。作者采用：

- 每题先采样 n 个候选，论文中常用 $n = 200$ 。
- 运行隐藏测试，数出 c 个正确候选。
- 对任意 $k \leq n$ ，估计从 n 个候选中抽 k 个时至少抽到一个正确候选的概率。

公式可以写成：

$$\text{pass@k for one task} = 1 - C(n - c, k) / C(n, k)$$

含义是：

- $C(n, k)$ 是从 n 个候选里抽 k 个的总方式数。
- $C(n - c, k)$ 是只从失败候选里抽 k 个的方式数。
- 二者相除是“抽到的 k 个全都失败”的概率。
- 1 减掉它，就是“至少有一个正确”的概率。

如果 $n = 10, c = 1, k = 5$ ：

$$\begin{aligned}\text{pass@5} &= 1 - C(9, 5) / C(10, 5) \\ &= 0.5\end{aligned}$$

因为 10 个候选里只有 1 个正确，抽 5 个候选时包含这个正确候选的概率就是 5/10。

如果 $n = 10, c = 3, k = 1$ ：

$$\begin{aligned}\text{pass@1} &= 1 - C(7, 1) / C(10, 1) \\ &= 0.3\end{aligned}$$

也就是普通的正确样本比例。

论文还给出数值稳定写法，不直接算巨大的组合数，而是逐项乘失败概率。本仓库现在对应实现为：

```
def estimate_pass_at_k(n: int, c: int, k: int) -> float:
    if not (0 <= c <= n):
        raise ValueError("c must be between 0 and n")
    if not (1 <= k <= n):
```

```

        raise ValueError("k must be between 1 and n")
    if n - c < k:
        return 1.0

    fail_prob = 1.0
    for i in range(n - c + 1, n + 1):
        fail_prob *= 1.0 - k / i
    return 1.0 - fail_prob

```

这段代码对应论文 Figure 3 的核心思想。

7. 为什么不能偷懒用 naive 公式

你可能会想:

$$\text{pass}@k = 1 - (1 - \text{pass}@1)^k$$

这个公式看起来合理，因为它像是“每次独立失败概率相乘”。但论文 Appendix A 专门说明，用有限样本的 empirical pass@1 代进去会产生偏差，通常会低估。

更细地说:

- 真实世界里，如果你知道某题一次采样正确的真实概率 p ，那么 $1 - (1-p)^k$ 是独立采样下至少成功一次的概率。
- 但评测时你不知道 p ，只知道在 n 个样本里观察到 c 个正确。
- 直接把 c/n 当 p 再代入，会把估计问题和真实概率问题混在一起。
- 论文的估计器把“从已观察的 n 个样本里无放回抽 k 个”作为估计对象，能保持无偏。

新手最容易犯的错是把 pass@k 当成 pass@1 的平滑版本。实际上 pass@k 是对模型采样分布、搜索预算和候选筛选能力的联合描述。

8. 评测管线图

论文的完整评测管线可以画成:

```

[HumanEval problem]
signature + docstring + hidden tests
    ->
[Prompt assembly]
header + function signature + docstring
    ->
[Codex sampling]
n completions with top-p sampling
    ->
[Stop sequence]
stop at class/def/comment/if/print boundaries
    ->
[Sandbox execution]
run each completion against hidden unit tests
    ->
[Count c]
c correct out of n samples
    ->

```

```
[pass@k estimator]
1 - C(n-c,k)/C(n,k)
    ->
[Aggregate]
average over 164 tasks
```

注意中间有两个评测设计很关键:

1. 采样时用 stop sequences, 避免模型继续生成额外函数或无关语句。
2. 执行时用 sandbox, 因为模型输出和 GitHub 代码都不能被默认信任。

9. sandbox 为什么不是小事

论文第 2.3 节强调, 执行模型生成的代码有安全风险。原因有三类:

- 公开 GitHub 仓库里可能存在恶意代码。
- 模型生成的代码可能意外访问文件、网络或系统资源。
- 单元测试需要真实执行代码, 所以评测框架必须处理不可信程序。

作者的 sandbox 目标是防止生成程序:

- 修改宿主环境。
- 获得持久化能力。
- 访问敏感资源。
- 向外部网络泄漏数据。

论文选择 gVisor 作为主要隔离组件, 并用 eBPF 防火墙限制入站和出站连接。这里的重点不是“一定要用 gVisor”, 而是:

execution-based eval 不是只写几行 exec 就结束; 真实评测必须把安全边界当成评测协议的一部分。

本仓库的 `safe_exec` 是教学 toy 版, 只做表层 forbidden pattern 检查和受限 builtins。它足够演示 HumanEval 机制, 但不能当真实 sandbox 使用。

10. 模型和训练数据

论文研究的是 Codex: 在公开 GitHub 代码上 fine-tune 的 GPT 类语言模型。

训练数据来自 2020 年 5 月收集的 GitHub 公共仓库:

- 约 5400 万个 public software repositories。
- 179 GB unique Python files, 文件大小小于 1 MB。
- 过滤自动生成文件、过长行、非字母数字比例异常等。
- 最终 Python 数据约 159 GB。

训练方法:

- 从 GPT 模型家族出发 fine-tune。作者发现从 GPT 初始化不一定提高最终效果, 但收敛更快。
- 训练 100 billion tokens。
- Adam optimizer, $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 1e-8$ 。
- weight decay = 0.1。
- 175 step linear warmup, 然后 cosine learning rate decay。
- 使用基于 GPT-3 文本 tokenizer 的 code lexer, 并增加 whitespace run tokens, 使代码约少用 30% token。

这里有一个重要动机: 代码中缩进和空白很频繁。如果 tokenizer 对空白低效, 同样上下文长度里能容纳的有效代码就更少, 训练和推理成本也更高。

11. token 级别怎么想

Codex 本质上还是自回归语言模型。对 HumanEval prompt, 它做的是:

```
input tokens:
    [def, count, _, vowels, (, s, ), :, newline, indent, docstring, ...]
```

```
model output:
    logits over vocabulary at each next position
```

```
sampling:
    choose next token from distribution
    append token
    repeat until stop sequence
```

张量层面可以这样想:

```
prompt_ids:      [seq_len]
hidden_states:    [seq_len, d_model]
logits:           [seq_len, vocab_size]
sampled_tokens:   [new_seq_len]
completion_code:  string decoded from sampled_tokens
```

HumanEval 评测关心的不是 logits 本身, 而是 logits 经过采样变成代码后, 代码执行是否满足测试。

这就是代码模型评测的一个转折:

```
language modeling loss
    measures token prediction
```

```
functional correctness
    measures executed program behavior
```

二者相关, 但绝不等价。

12. 主结果: Codex 在 HumanEval 上发生了什么

论文的摘要和主表给出一组很有历史意义的数字。

HumanEval pass@1:

- GPT-3 类模型在这个任务上接近 0。
- GPT-J 6B 达到约 11.4% 或 11.62%。
- Codex-12B 达到 28.8% 或 28.81%。
- Codex-S-12B 单样本达到约 37.7%。

HumanEval pass@100:

- Codex-12B 表 1 为 72.31%。
- 论文摘要报告重复采样能把解决率推到约 70.2%。
- 图 1 中 Codex-S-12B 在 oracle test selection 下约 77.5%。

这组数字说明两件事:

1. 代码 fine-tuning 明显改变了模型能力。不是模型大就自动会写代码，数据分布很重要。
2. 多采样收益极大。模型分布里已经包含很多正确程序，但 pass@1 只能看到第一口样本。

如果你用一句话解释 Figure 1:

Codex 学会了把 docstring 转成可运行函数；Codex-S 通过更接近 HumanEval 的监督数据进一步提升；多采样加测试筛选能把可用正确程序从模型分布里捞出来。

13. 温度和采样数量

论文有一个很实用的观察: 不同 k 需要不同 sampling temperature。

对 679M 模型:

- pass@1 的最优温度约是 $T = 0.2$ 。
- pass@100 的最优温度约是 $T = 0.8$ 。

原因很直观:

- 如果只采一个样本，你希望输出集中、保守、高概率。
- 如果要采很多样本，你希望分布更分散，覆盖更多可能程序。
- pass@k 只关心候选里有没有至少一个正确，所以多样性变得有价值。

这对你用 agent 学习也很重要。调代码时，不同目标对应不同采样策略:

- 要稳定复现: 低温度。
- 要探索候选解法: 高一点温度。
- 要用测试筛选: 可以增加采样数量。
- 要直接交付用户: 不能只看 pass@100，必须考虑可筛选性和安全性。

14. 候选排序: 没有测试时怎么办

pass@k 的 oracle 版本假设你可以运行测试，知道哪个候选正确。但真实 autocomplete 场景中，用户可能没有测试，IDE 也不可能总是执行候选。

论文因此研究了从 k 个样本里选一个的方法:

- 随机选一个。
- 选 sum log probability 最大的。
- 选 mean token log probability 最大的。
- 用 docstring generation 做 back-translation ranking。
- oracle: 选通过隐藏测试的样本。

结果里，一个重要发现是:

- mean token log probability 比随机好。
- sum log probability 有时甚至比随机差，因为它偏好短样本。
- back-translation 比随机好，但不如 mean log probability，而且容易 overfit。
- 有测试时，测试筛选最强。

论文在图 1 提到，Codex-S-12B 生成 100 个样本时:

- oracle test selection 可解决约 77.5%。
- 选 mean log probability 最好的样本可解决约 44.5%。

这说明“模型知道什么”和“你能选出什么”是两个问题。对 agent 来说，这正是 execution feedback 的价值。

15. BLEU 为什么靠不住

论文用 Codex-12B 的 HumanEval 样本比较了正确解和错误解的 BLEU 分布。结论是两者大量重叠。

这意味着：

- 有些错误程序和参考答案很像。
- 有些正确程序和参考答案不像。
- BLEU 提升不一定代表功能正确率提升。

一个典型错误是边界条件：

```
def is_prime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True
```

这段代码看起来像 prime check，也可能和参考答案有较高 overlap，但它会把 $n = 1$ 判成 True。功能正确性评价会抓住这个错误，BLEU 不一定能抓住。

16. 与 GPT-Neo、GPT-J、TabNine 的对比

论文把 Codex 与当时的开放模型和代码补全系统比较。

关键数字：

- GPT-Neo 125M: pass@1 0.75%, pass@100 2.97%。
- GPT-Neo 1.3B: pass@1 4.79%, pass@100 16.30%。
- GPT-Neo 2.7B: pass@1 6.41%, pass@100 21.37%。
- GPT-J 6B: pass@1 11.62%, pass@100 27.74%。
- TabNine: pass@1 2.58%, pass@100 7.59%。
- Codex-300M: pass@1 13.17%, pass@100 36.27%。
- Codex-12B: pass@1 28.81%, pass@100 72.31%。

你应该从这些数字里读出：

1. GitHub 代码 fine-tuning 很关键。GPT-J 6B 大约相当于 Codex-300M 的 HumanEval 表现。
2. 模型规模仍然重要。Codex 从 12M 到 12B，pass@1 和 pass@100 都平滑上升。
3. pass@100 的上升幅度比 pass@1 更显著，说明大模型更常把正确解放进采样分布里。

这不是简单的“参数越大越好”，而是：

训练数据分布 + 模型规模 + 采样策略 + 执行筛选

共同决定代码生成表现。

17. APPS 实验说明了什么

APPS 是另一个代码挑战数据集，包含：

- 5000 train examples。
- 5000 test examples。
- 题型更接近完整竞赛程序，常常需要读 stdin、写 stdout。
- 题目包含输入输出示例。

这与 HumanEval 有明显差异：

- HumanEval 是单函数补全。
- APPS 更像完整程序综合。
- HumanEval 比较短、局部、结构稳定。
- APPS 难度更高，时间复杂度也更重要。

论文在 APPS 上给 Codex 一个 1-shot formatting hint: 把题目中的一个 input/output 示例追加进 docstring。然后采样最多 1000 个候选，并区分：

- raw pass@k: 直接看隐藏测试通过率。
- filtered pass@k: 先用公开的 3 个 input/output examples 过滤，再算 pass。
- timeout: 有些程序逻辑可能对，但算法太慢，隐藏测试超时。

重要数字：

- GPT-Neo 2.7B raw pass@1 在 introductory/interview/competition 分别约 3.90%、0.57%、0%。
- 1-shot Codex raw pass@1000 分别约 25.02%、3.70%、3.23%，括号中的 timeout-inclusive 数更高。
- 1-shot Codex filtered pass@1 分别约 22.78%、2.64%、3.04%。

APPS 实验的意义不是说 Codex 已经解决竞赛编程。恰恰相反，它说明：

- HumanEval 的好成绩不能直接外推到更难、更长、更完整的任务。
- 多采样和公开样例过滤有用。
- 代码评测必须考虑效率和超时，不只是功能逻辑。

这也是后来 LiveCodeBench 和 SWE-bench 要继续往前走的原因。

18. Codex-S: 为什么监督微调有用

Codex 训练在 GitHub Python 文件上。GitHub 代码分布非常杂：

- 类定义。
- 配置脚本。
- 数据文件。
- 工具函数。
- 测试代码。
- package glue code。
- standalone functions。

HumanEval 要求的是“从 docstring 生成 standalone function”。这个分布和普通 GitHub 文件不完全一致。

所以作者构造更接近 HumanEval 的监督数据，训练 Codex-S。数据来源有两类。

第一类：竞赛和面试网站。

- 题目自包含。
- 有良好问题描述。
- 通常有隐藏测试。
- 作者整理了约 10000 个问题。

第二类：continuous integration traces。

- 从开源项目 CI 运行中追踪函数输入输出。
- 用 `sys.setprofile` 收集函数调用。
- 把输入输出变成函数级单元测试。
- 作者最后收集约 40000 个问题。

然后还要过滤质量:

- 有些 prompt 欠规范, 正确解可能被测试误杀。
- 有些函数有状态或非确定性。
- 作者让 Codex-12B 对每个候选问题采样 100 个解, 如果没有解通过, 就认为题目可能太难或不清楚, 过滤掉。
- 反复验证以去掉状态性和非确定性问题。

训练上, Codex-S 对 reference solution 做 negative log-likelihood, 并 mask prompt token 的 loss。学习率比 Codex fine-tuning 小 10 倍, 训练到 validation loss plateau, 少于 10B tokens。

结果:

- Codex-S 平均比 Codex pass@1 高约 6.5 个百分点。
- pass@100 平均高约 15.1 个百分点。
- Codex-S 对 pass@1 的最佳温度接近 0, 对 pass@100 的最佳温度接近 1。
- 图 10 显示 Codex-S 在 HumanEval 上更 parameter efficient。

动机可以这样总结:

预训练让模型学代码分布; 监督微调把模型输出分布拉向 benchmark 需要的交互格式。

19. Codex-D: 反过来从代码写 docstring

论文第 5 节训练了一个反向模型 Codex-D: 给函数签名和代码体, 让模型生成 docstring。

为什么要做这个? 一个安全动机是:

- 如果模型能描述代码意图, 就可能帮助人类理解生成代码。
- 也可以用 back-translation: 生成代码后, 让 docstring 模型评估这段代码有多像原始规格。

训练数据来自前面 Codex-S 的问题:

```
signature + reference solution -> docstring
```

评价方式不是自动测试, 因为 docstring 没有像代码一样的执行判据。作者手工评分:

- 164 个 HumanEval task。
- 每题 10 个 docstring samples。
- 共 1640 个样本。
- 如果 docstring 能唯一且准确说明代码体, 就算正确。
- 直接复制代码体不算正确。

结果:

- Codex-S-12B 在同温度下 pass@1 约 32.2%, pass@10 约 59.5%。
- Codex-D-12B pass@1 约 20.3%, pass@10 约 46.5%。

Codex-D 的失败模式包括:

- 漏掉关键细节, 例如小数位要求。
- 过度依赖函数名, 编造与函数体不匹配的问题。
- 生成奇怪或低质量的 docstring。

用 Codex-D 做 sample ranking 时, back-translation 比随机好, 但不如 mean log probability, 而且过拟合快。

对 agent 学习的启发是: 自我解释和反向描述有帮助, 但不能替代执行测试。

20. 失败模式: 长链操作和变量绑定

论文第 6 节非常重要, 因为它提醒我们不要被 pass@100 的高数字迷惑。

Codex 的主要限制包括:

- 训练样本效率低。它看过大量 GitHub 代码, 但仍只解决一部分 HumanEval。
- 对长链操作不稳。
- 对变量和操作的绑定容易错。
- 可能调用未定义函数、变量或属性。
- 对更高层、更系统级的规格理解差。

作者构造了一个 synthetic string task 数据集。每个 docstring 由 13 种基本字符串操作组合而成, 例如:

- 删除所有字母 e。
- 把空格替换成感叹号。
- 转成小写。
- 删除每三个字符。
- 反转单词顺序。

随着 docstring 中串联的 building blocks 增加, Codex-12B 的 pass rate 大约每多一个组件下降 2 到 3 倍。

这说明模型不是像人类程序员那样掌握了可组合算法。它在短规格上能工作, 但操作链越长, 绑定、顺序和中间状态越容易乱。

论文还给了变量绑定失败例子: docstring 要求对多个变量分别加减, 再返回乘积, 但模型漏掉某个变量操作, 或者返回了错误中间量。

这对今天的 agent 仍然关键: 短函数 benchmark 高分, 不等于多文件、多约束、多步骤软件任务可靠。

21. 安全和 broader impacts

这篇论文的第 7 节很长, 不是附带闲聊, 而是代码生成研究里很早的一次系统 hazard analysis。

主要风险包括:

过度依赖。

用户可能相信看起来很合理的代码, 尤其是新手。代码可能语法正确、风格自然, 但边界条件错、安全性差或不符真实意图。

misalignment。

论文把一种情况叫作 intent misalignment: 模型有能力做对, 但因为训练分布或 prompt 中的错误暗示, 反而生成不符合用户意图的代码。论文 Figure 12 显示, 当 prompt 中有微妙 bug 时, Codex 会生成更差代码, 而且这个 gap 随模型规模变大。

偏见和表示问题。

模型可能在注释、变量名、分类逻辑或生成结构中反映训练数据中的刻板印象。

经济和劳动影响。

代码生成可能提升生产率, 也可能改变软件工程岗位结构、开源维护生态和包作者可见性。

安全风险。

Codex 可能生成漏洞代码, 也可能被滥用于网络攻击。论文认为当时的模型未必显著降低 malware 开发门槛, 但更强模型需要持续研究缓解措施。

数据和供应链风险。

公开代码中可能有敏感信息、恶意样本或被投毒数据。模型训练在 public repositories 上，不代表训练数据可信。

法律和知识产权。

论文讨论了训练公共代码、生成代码和复制训练片段之间的法律问题，并引用了早期关于 Copilot rote memorization 的观察：完全相同生成很少见，低于 0.1%，且常是常见表达。

论文提出的缓解方向包括：

- 文档和 UI 明确提醒模型限制。
- 要求人类 review。
- 输出过滤和内容控制。
- API 服务中的用例限制、监控、rate limiting。
- 在高风险场景中谨慎部署。

你读这一节时要意识到：execution-based eval 只测“能不能通过测试”，不能自动测“这段代码是否安全、合规、可维护、符合用户真正意图”。

22. 论文证据链条

这篇论文的证据链不是一个表格，而是一组互相支撑的实验。

第一条证据：HumanEval 主结果。

- GPT 类模型接近 0。
- Codex-12B pass@1 约 28.8%。
- Codex-S-12B pass@1 约 37.7%。
- 多采样显著提升 pass@100。

证明了代码 fine-tuning 和 task-distribution supervised fine-tuning 对函数级正确性有帮助。

第二条证据：scaling。

- Codex held-out Python loss 随模型规模呈平滑 power law。
- pass@1 和 pass@100 也随规模上升。

证明代码模型能力不是孤立偶然点，而是和规模趋势相关。

第三条证据：temperature and sampling。

- pass@1 适合低温。
- pass@100 适合较高温。
- 多样性对多采样指标有价值。

证明评测指标会改变推理策略。

第四条证据：sample ranking。

- mean log probability ranking 优于随机。
- sum log probability 可能有长度偏置。
- back-translation 有帮助但不如 mean log probability。
- oracle test selection 最强。

证明“生成候选”和“选择候选”是两个独立问题。

第五条证据：BLEU overlap。

- 正确和错误解的 BLEU 分布重叠。

证明 match-based metric 不足以反映功能正确性。

第六条证据: APPS。

- 更完整、更难的程序任务上, Codex 仍远未可靠。
- public examples filtering 和大量采样有帮助。
- 超时显示效率也是正确性的一部分。

证明 HumanEval 不能代表全部代码能力。

第七条证据: limitations 和 synthetic tasks。

- 长链操作导致性能快速下降。
- 变量绑定和系统级理解不足。

证明模型有结构性短板。

23. 这篇论文没有证明什么

为了不误读, 需要明确它没有证明:

- Codex 能可靠完成真实软件工程。
- HumanEval 能覆盖所有代码能力。
- pass@100 高就等于用户体验好。
- 单元测试通过就等于程序安全。
- 代码模型理解了算法的可组合结构。
- sample ranking 已经解决了没有测试时的选择问题。
- 训练在公开 GitHub 上没有法律、隐私或供应链问题。

这篇论文的诚实之处在于, 它不仅展示能力, 也展示限制和风险。你应该学习这种读法: 指标上升是证据, 但不是世界模型的终点。

24. 和本仓库代码的对应关系

本模块文件:

- learning/agent-code-eval/src/humaneval_runner.py
- learning/agent-code-eval/src/common.py
- learning/agent-code-eval/src/mbpp_runner.py
- learning/agent-code-eval/src/livecodebench_mock.py
- learning/agent-code-eval/src/swebench_mock.py
- learning/agent-code-eval/src/webarena_mock.py
- learning/agent-code-eval/src/bfcl_runner.py
- learning/agent-code-eval/src/mini_agent.py

common.py 对应论文里的基础执行组件:

- extract_code: 从模型文本里提取代码块。
- safe_exec: toy sandbox, 执行代码和测试。
- CodeResult: 保存预测代码、通过状态和错误。
- pass_rate: 聚合通过率。

humaneval_runner.py 对应 HumanEval:

- _TASKS: toy 版函数题。

- build_prompts: 组装函数签名和 docstring prompt。
- run_humaneval: 调模型、抽代码、执行测试。
- estimate_pass_at_k: 论文 pass@k 无偏估计器。
- run_passk: 多采样并聚合 pass@1/pass@k。

mbpp_runner.py 和 livecodebench_mock.py 对应后续 code benchmark:

- MBPP 更像基础自然语言编程题。
- LiveCodeBench 强调更新、更难、更接近竞赛的题目。

swebench_mock.py 对应从函数生成走向 issue repair:

- 输入不只是 docstring, 而是 repository file 和 issue。
- 输出不是函数体, 而是修复后的文件。
- 评价是隐藏测试是否通过。

bfcl_runner.py 和 webarena_mock.py 对应 agent 时代:

- BFCL 评测 tool call 名称和参数是否正确。
- WebArena 评测动作序列是否达到目标状态。

mini_agent.py 把这些合在一起, 形成一个小型 capstone:

```
HumanEval
LiveCodeBench
SWE-Bench mock
WebArena mock
BFCL mock
    ->
multi-benchmark score
```

这就是从 HumanEval 到 agent eval 的最小路线图。

25. 本仓库最小实验 1: pass@k 手算

先在纸上算:

```
n = 10
c = 1
k = 5
```

```
pass@5 = 1 - C(9,5) / C(10,5)
        = 0.5
```

然后读代码:

```
from humaneval_runner import estimate_pass_at_k

print(estimate_pass_at_k(10, 1, 5))
```

你应该得到 0.5。

再试:

```
print(estimate_pass_at_k(10, 3, 1))
print(estimate_pass_at_k(10, 3, 10))
```

理解:

- $k = 1$ 时就是单样本正确比例。
- $k = n$ 且 $c > 0$ 时一定能抽到正确样本。

26. 本仓库最小实验 2: 隐藏测试抓边界条件

打开 `humaneval_runner.py` 的 `_TASKS`, 任选一个任务, 故意写一个看起来合理但边界错的 mock answer. 例如:

```
def is_even(n):
    return n > 0 and n % 2 == 0
```

它会在 $n = 0$ 的测试上失败。这个实验要你看:

- 文本看起来像答案不够。
- 参考答案相似不够。
- hidden unit tests 才是功能正确性的近似判据。

27. 本仓库最小实验 3: pass@1 和 pass@k 的差别

你可以构造一个非确定 mock model, 让它有时返回正确答案, 有时返回错误答案。概念上:

```
answers = [
    "wrong code",
    "wrong code",
    "correct code",
    "wrong code",
]
```

如果第一个样本错, pass@1 可能失败; 但 k 个样本里出现正确候选, pass@ k 会变高。

这就是论文最重要的直觉:

模型分布里存在正确程序, 不代表 greedy 或第一个样本会拿到它。

28. 本仓库最小实验 4: 从 HumanEval 到 SWE-Bench

对比两个 prompt:

HumanEval:

Complete this function from signature and docstring.

SWE-Bench mock:

Here is a repository file.

Here is an issue.

Provide the fixed file contents.

差异:

- HumanEval 是局部函数综合。
- SWE-Bench 是仓库修复。
- HumanEval 的隐藏测试是函数行为。
- SWE-Bench 的隐藏测试是 patch 是否修好 issue。

这说明 HumanEval 是起点, 不是终点。

29. 如果你要用 AI agent 学这篇

不要让 agent 只总结。正确用法是让 agent 逼你做三件事:

1. 用自己的话解释指标。
2. 在代码里找到指标实现。
3. 用 toy case 验证一个公式或失败模式。

推荐提示词:

我正在读《Evaluating Large Language Models Trained on Code》。

请一次只问我一个问题，不要直接给总结。

问题必须覆盖:

1. HumanEval 为什么要手写。
2. functional correctness 为什么优于 BLEU。
3. pass@k 的无偏估计公式。
4. 为什么不同 k 需要不同 temperature。
5. Codex-S 为什么比 Codex 更适合 HumanEval。
6. 论文的安全风险分析说明了什么。

每次我回答后，请指出哪里含糊，并要求我把答案对应到本仓库某个源码函数。

闭卷复述目标:

这篇论文把代码模型评测从 token 相似度推进到可执行功能正确性。

作者发布 HumanEval，用 164 个手写 Python 函数题和隐藏单元测试评估模型。

Codex 在 GitHub Python 上 fine-tune 后，HumanEval pass@1 明显超过 GPT 类模型，多采样下 pass@100 大概 pass@k 用从 n 个样本中 c 个正确的无偏组合估计，而不是简单套 pass@1。

论文还说明温度、样本选择、监督微调、APPS 泛化、长链操作失败、变量绑定失败和安全风险。

它的重要性在于建立了 execution-based code evaluation 的基本语言，但 HumanEval 不能代表真实软件工程的

30. 对现在的意义

今天看这篇论文，要把它放在三条发展线里。

第一条: code model benchmark。

HumanEval 之后，社区不断补它的缺点:

- MBPP 补更多基础 Python 题。
- APPS 补竞赛程序。
- BigCodeBench 补更真实库使用。
- LiveCodeBench 补新题和污染问题。
- SWE-bench 补真实 issue 和 patch。

第二条: agent evaluation。

HumanEval 的模式是:

generate code -> execute tests -> score

agent 的模式扩展成:

plan -> act -> use tools -> modify environment -> observe -> retry -> score final state

但核心仍然是 execution feedback。没有执行反馈，agent 很容易只是在生成看起来合理的步骤。

第三条: safe deployment。

代码生成比普通文本生成更容易进入真实系统。一个错误函数可能导致：

- 数据损坏。
- 安全漏洞。
- 服务中断。
- 隐私泄漏。
- 用户过度信任。

所以评测必须同时看：

- 功能正确性。
- 鲁棒性。
- 安全性。
- 可维护性。
- 人类 review 流程。

31. 读完必须能回答

你应该能闭卷回答：

1. 为什么 BLEU 不适合做代码生成主指标？
2. HumanEval 每个样本包含哪些部分？
3. pass@k 的公式是什么，n、c、k 分别代表什么？
4. 为什么 $1 - (1 - c/n)^k$ 不是论文推荐估计器？
5. 为什么 pass@1 和 pass@100 的最佳 temperature 不同？
6. Codex 和 Codex-S 的区别是什么？
7. APPS 实验揭示了 HumanEval 的什么边界？
8. 长链字符串任务说明了模型哪种能力不足？
9. 为什么真实 HumanEval runner 需要 sandbox？
10. 本仓库哪个函数对应论文的 pass@k？
11. 本仓库哪个文件展示 toy HumanEval？
12. 从 HumanEval 到 SWE-Bench，评测对象发生了什么变化？

如果这些问题能答出来，再去读后续 benchmark 会轻松很多。

32. 你下一步该怎么学

建议学习节奏：

1. 先读本 guide 到第 8 节，手算 pass@k。
2. 打开 humaneval_runner.py，找到 estimate_pass_at_k 和 run_humaneval。
3. 跑 `python learning/agent-code-eval/src/tests/test_agent.py`。
4. 故意写一个边界条件错误的 mock answer，观察测试失败。
5. 再读第 16 到 22 节，整理“主结果、APPS、Codex-S、失败模式、安全”五张卡片。
6. 最后打开 mini_agent.py，理解为什么 agent eval 是 HumanEval 思想的扩展。

这篇学完以后，你对代码模型评测的判断标准应该从“模型会不会写一段像样代码”升级为：

它能不能在给定规格下生成可执行程序？

是否通过隐藏测试？

多采样是否显著提高成功率？

没有测试时如何选择候选？

任务是否被 benchmark 充分覆盖？

代码是否安全、可维护、符合用户真实意图？

这才是 HumanEval 这篇论文真正要带进你脑子里的东西。